TRABAJO FINAL DE MICROSERVICIOS

1. Resumen del Sistema

El presente proyecto implementa un sistema distribuido para la gestión de pedidos de compra.

La arquitectura se compone de tres microservicios autónomos (Pedidos, Catálogo y Pagos), un Message Broker (RabbitMQ) y un servicio de terceros simulado (Pasarela de Pagos). Cada microservicio expone una API REST, cuenta con su propia base de datos aislada para garantizar un bajo acoplamiento, e implementa autenticación mediante JWT. Todo el ecosistema está orquestado y empaquetado utilizando Docker y Docker Compose para asegurar la reproducibilidad del entorno.

El núcleo transaccional del sistema resuelve el problema de la consistencia de datos a través de bases de datos independientes mediante la implementación del patrón Saga basado en Orquestación, garantizando transacciones distribuidas seguras con capacidades de compensación ante fallos.

2. Decisiones Clave y Compromisos (Trade-offs)

- **Saga Orquestada vs. Coreografiada:** Se optó por un enfoque de orquestación donde el microservicio de Pedidos actúa como coordinador central.
    - Compromiso: Aunque esto centraliza la lógica de negocio y genera un acoplamiento lógico hacia el orquestador ofrece una visibilidad completa del estado de la transacción (pendiente, confirmado, rechazado), lo cual es crítico para el negocio y facilita el monitoreo, a diferencia de la coreografía donde el flujo emergente es más difícil de rastrear.
- **Comunicación Asíncrona (RabbitMQ) para Pagos:**
    - Compromiso: En lugar de realizar llamadas REST síncronas hacia el servicio de Pagos, se implementaron colas durables. El trade-off asumido es sacrificar la inmediatez a cambio de una consistencia eventual y alta disponibilidad, requiriendo mecanismos de idempotencia (correlation_id) para evitar dobles cobros si los mensajes se reentregan.
- **Circuit Breaker para Aislamiento de Fallos externos:**

- Compromiso: La integración con la pasarela externa asume que la red es inherentemente inestable. Se utilizó la librería opossum para abrir el circuito si el 50% de las transacciones fallan o superan los 4000ms. Esto sacrifica la retención de posibles pagos legítimos durante estados degradados para proteger los recursos internos (memoria y CPU) del servicio de Pagos mediante la técnica de "fail-fast".

3. Modos de Fallo Probados

Durante el desarrollo y estabilización del sistema, se probaron y mitigaron los siguientes escenarios de fallo:

1. **Fallo o Latencia Extrema en la Pasarela de Pagos (Simulado vía Stub):** Al inyectar un 30% de tasa de error y hasta 5 segundos de latencia, el Circuit Breaker interrumpe la petición. El sistema de Pagos captura este error, emite un evento de rechazo a RabbitMQ, y el orquestador (Pedidos) ejecuta el rollback exitosamente, invocando el endpoint de compensación (/v1/catalogo/liberar) para devolver el stock reservado.
2. **Fallo de Dependencias al Inicio (Condiciones de Carrera en Docker):** Se observó que RabbitMQ requería tiempos prolongados de inicialización (Erlang VM), lo que provocaba el colapso en cascada de los microservicios de Node.js al recibir rechazos de conexión (ECONNREFUSED).
    - Mitigación: Se implementaron healthchecks exhaustivos en Docker Compose, incrementando el start_period a 60 segundos y utilizando depends_on: condition: service_healthy para garantizar un arranque determinista.
3. **Transacciones Duplicadas:** Se simuló el reenvío de peticiones. La base de datos captura el error gracias a la restricción UNIQUE sobre el correlation_id, garantizando la idempotencia transaccional.

4. Limitaciones del Sistema

- **Falta de un API Gateway:** Actualmente, los clientes deben conocer la topología de red para consumir el servicio de Pedidos. Un API Gateway centralizaría el enrutamiento, la terminación SSL y la validación de tokens.
- **Gestión Centralizada de Logs:** Si bien se utiliza Pino para logs estructurados e inyección de correlation_id, el sistema carece de un agregador externo (como el stack ELK o Grafana Loki), lo que obliga a revisar los contenedores individualmente mediante la terminal.

- **Bloqueo de Base de Datos Síncrono:** La reserva de stock en Catálogo no utiliza bloqueos optimistas avanzados ni particionamiento, lo que podría generar problemas de concurrencia bajo cargas masivas.

5. Declaración de Uso de IA

Se declara el uso del asistente de inteligencia artificial Gemini de Google como herramienta de apoyo al desarrollo durante este examen.